

CHAPITRE 2

Créer des microservices

1. Microservice versus API REST

2. Création d'une application en microservices avec Node.js
3. Communication entre microservices avec Rabbitmq
4. Mise en place d'un reverse proxy en utilisant Nginx



2. Créer des microservices

Microservice versus API REST

Qu'est ce qu'un API?

- API signifie **A**pplication **P**rogramming **I**nterface;
- C'est une **interface** qui permet à deux applications de communiquer;
- Un logiciel est utilisé par des **utilisateurs** via une **interface utilisateur**. Cependant, les gens ne sont pas les seuls à utiliser les logiciels. Le logiciel est également utilisé par d'autres applications. Cela nécessite un autre type d'interface qui s'appelle une interface de programmation d'application, ou en abrégé **API**;
- Ils sont considérés comme des **portes** qui permettent aux **développeurs** d'interagir avec une application;
- C'est une façon d'effectuer des requêtes sur un composant.

Exemples:

- API Google Map permet aux développeurs de générer et d'afficher des cartes personnalisables sur leur site web.

2. Créer des microservices

Microservice versus API REST

Tableau de comparaison

	API	Microservice
Définition	Il s'agit d'un ensemble de méthodes prédéfinies facilitant la communication entre différents composants.	C'est une architecture qui consiste à diviser les différentes fonctions d'une application en composants plus petits et plus agiles appelés services.
Concept	Les API offrent un moyen simple de se connecter, de s'intégrer et d'étendre un système logiciel.	L'architecture des microservices est généralement organisée autour des besoins en capacité et des priorités de l'entreprise;
Fonction	Les API fournissent une interface réutilisable à laquelle différentes applications peuvent se connecter facilement.	Les microservices s'appuient largement sur les APIs et les passerelles API pour rendre possible la communication entre les différents services.

CHAPITRE 2

Créer des microservices

1. Microservice versus API REST
- 2. Création d'une application en microservices avec Node.js**
3. Communication entre microservices avec Rabbitmq
4. Mise en place d'un reverse proxy en utilisant Nginx



2. Créer des microservices

Création d'une application en microservices avec Node.js



Raisons de créer des microservices avec NodeJS

Parmi tous les langages de programmation utilisés dans le développement d'applications de microservices, NodeJS est **largement utilisé** par une grande communauté des développeurs pour ses caractéristiques et les avantages qu'il apporte.

Voici quelques raisons pour lesquelles les microservices avec Node.JS sont le meilleur choix :

- **Il améliore le temps d'exécution** puisque NodeJS s'exécute sur le moteur V8 de Google, compile la fonction en code machine natif et effectue également des opérations CPU et IO à faible latence.
- **L'architecture événementielle** de NodeJS le rend très utile pour développer des applications événementielles.
- Les bibliothèques NodeJS prennent en charge **les appels non bloquants** qui continuent de fonctionner sans attendre le retour de l'appel précédent.
- Les applications créées avec NodeJS sont **évolutives**, ce qui signifie que le modèle d'exécution prend en charge la mise à l'échelle en attribuant la demande à d'autres threads de travail.

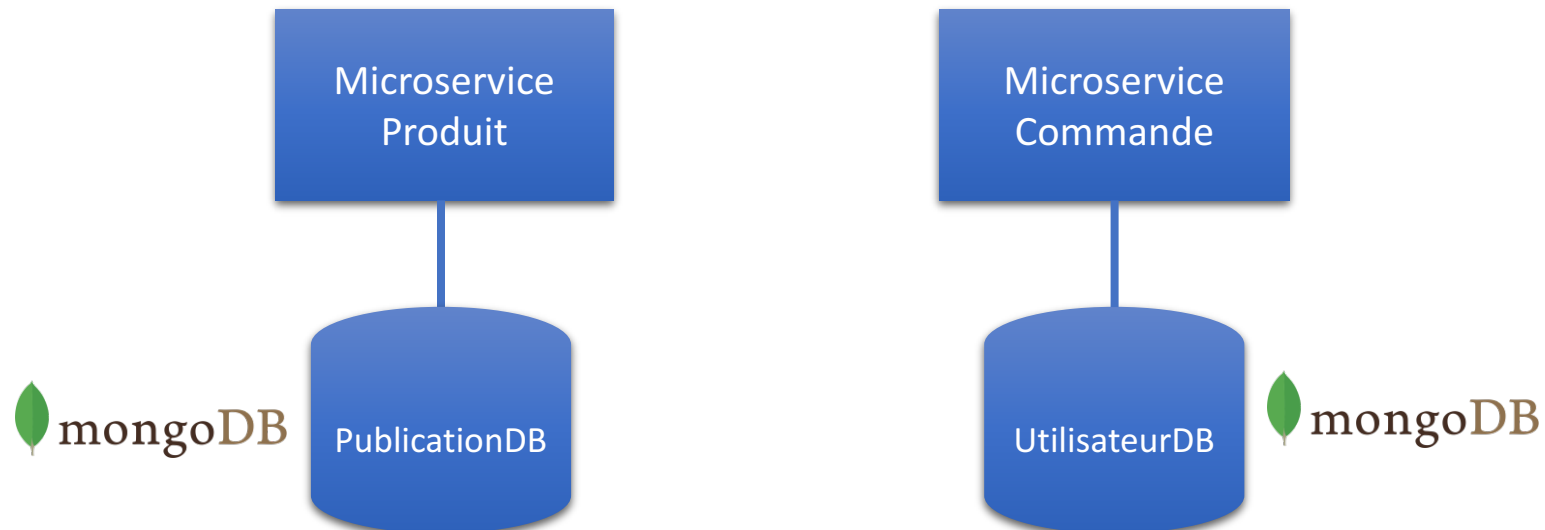
2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple

Un microservice est en outre défini par son architecture et sa portée. Par conséquent, d'un point de vue technique, implémenter un microservice via HTTP ne serait pas différent de l'implémentation d'une API NodeJS.

Exemple de microservices créés en Node js (Communication synchrone):

- Soit une simple partie d'application e-commerce gérant les produits et leurs commandes;
- L'application sera décomposée, dans un premier temps, en deux microservices : produit et commande;
- Chaque service aura sa propre base de données sous MongoDB
- Les deux services seront considérés comme deux applications séparées, chacune expose son API et écoute sur son propre port;



2. Créer des microservices

Création d'une application en microservices avec Node.js : Exemple

APIs à créer pour cet exemple :

Microservice
Produit

Le port d'écoute : **4000**

Méthode	URL	Signification
GET	localhost:4000/produit/acheter	Cherches les produits à acheter et retourne leur objets correspondants
POST	localhost:4000/produit/ajouter	Ajoute un nouveau produit à la base de données

Microservice
Commande

Le port d'écoute : **4001**

Méthode	URL	Signification
POST	localhost:4001/commande/ajouter	Ajoute une nouvelle commande à la base de données

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple



Données de l'exemple :

- Structure d'un objet produit :

_id : son identifiant

nom : son nom

description : des informations le décrivant

prix : sa valeur

created_at : la date de sa création (par défaut elle prend la date système)

- Structure d'un objet commande :

_id : son identifiant

produits: un tableau regroupant les ids des produits concernés par cette commande

email_utilisateur: l'adresse email de celui à qui appartient la commande

prix_total: le total des prix à payer pour finaliser la commande

created_at: la date de sa création (par défaut elle prend la date système)

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple

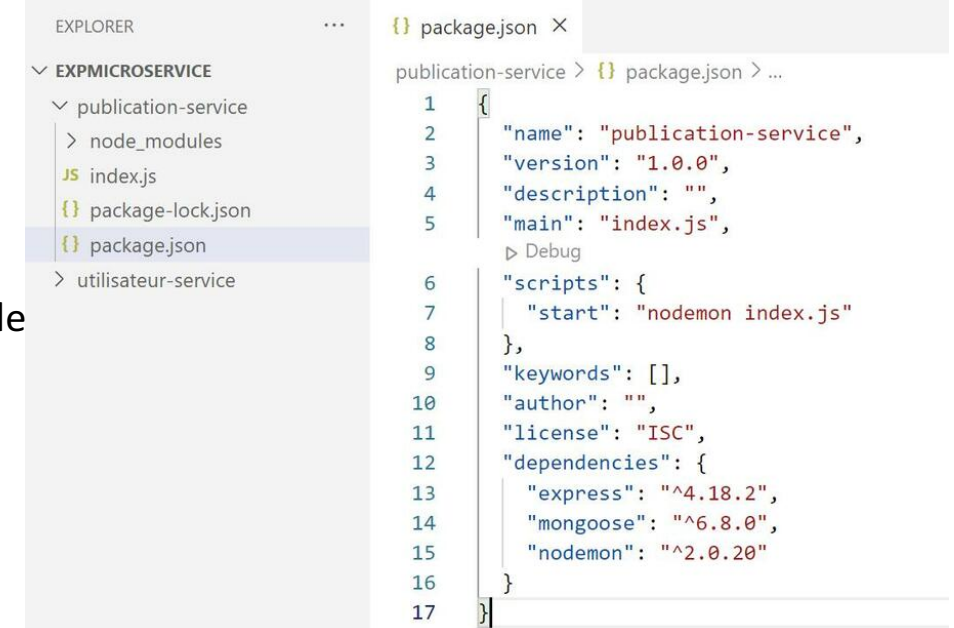
Exemple:

Afin de réaliser cet exemple, suivons les étapes ci-après:

1. Créer un dossier appelé expMicroservice;
2. Sous le dossier expMicroservice, créer deux sous dossiers : produit-service et commande-service;

3. Produit-service :

- a. Sous le dossier produit-service, ouvrir le terminal et exécuter les commandes suivantes :
`npm init -y`
`npm install express mongoose nodemon`
- b. Ajouter à la racine de ce dossier un fichier vide "index.js";
- c. Modifier la partie script du fichier package.json et remplacer le script existant par :
`"start": "nodemon index.js"`



The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the file structure of the 'EXPMICROSERVICE' project. It includes a 'publication-service' subdirectory containing 'node_modules', 'index.js', 'package-lock.json', and 'package.json'. The 'package.json' file is selected and its content is shown in the main editor area on the right. The 'package.json' file is for the 'publication-service' and contains the following JSON:

```
1 {
2   "name": "publication-service",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "start": "nodemon index.js"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "dependencies": {
13    "express": "^4.18.2",
14    "mongoose": "^6.8.0",
15    "nodemon": "^2.0.20"
16  }
17 }
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple



Exemple:

3. Produit-service :

d. Ajouter un fichier à la racine appelé « Produit.js »;

Ce fichier représentera le modèle « produit » à créer à base d'un schéma de données **mongoose**;

Ce schéma permettra par la suite de créer une collection appelée « produit » sous une base de données **MongoDB**;

```
const mongoose = require("mongoose");

const ProduitSchema = mongoose.Schema({
  nom: String,
  description: String,
  prix: Number,
  created_at: {
    type: Date,
    default: Date.now(),
  },
});

module.exports = Produit = mongoose.model("produit", ProduitSchema);
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple



e. Modifier le fichier index.js :

```
const express = require("express");
const app = express();
const PORT = process.env.PORT_ONE || 4000;
const mongoose = require("mongoose");
const Produit = require("../Produit");

app.use(express.json());

//Connection à la base de données MongoDB « publication-service-
db »
// (Mongoose créera la base de données s'il ne le trouve pas)
mongoose.set('strictQuery', true);
mongoose.connect(
  "mongodb://localhost/produit-service",
  {
    useNewUrlParser: true,
    useUnifiedTopology: true,
  },
  () => {
    console.log(`Produit-Service DB Connected`);
  }
);
app.post("/produit/ajouter", (req, res, next) => {
  const { nom, description, prix } = req.body;
  const newProduit = new Produit({
    nom,
    description,
```

```
    prix
  });

  //La méthode save() renvoie une Promise.
  //Ainsi, dans le bloc then(), nous renverrons une réponse de
  //réussite avec un code 201 de réussite.
  //Dans le bloc catch () , nous renverrons une réponse avec
  //l'erreur générée par Mongoose ainsi qu'un code d'erreur 400.
  newProduit.save()
    .then(produit => res.status(201).json(produit))
    .catch(error => res.status(400).json({ error }));
});

app.get("/produit/acheter", (req, res, next) => {
  const { ids } = req.body;
  Produit.find({ _id: { $in: ids } })
    .then(produits => res.status(201).json(produits))
    .catch(error => res.status(400).json({ error }));
});

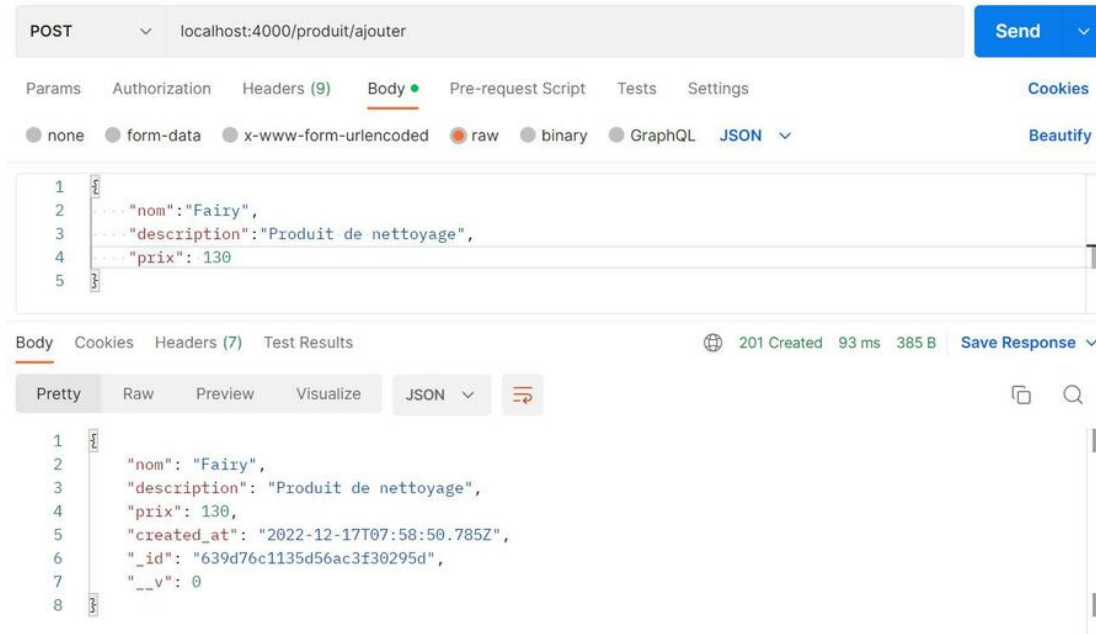
app.listen(PORT, () => {
  console.log(`Product-Service at ${PORT}`);
});
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple

4. Tester l'API exposée par le service : Publication-service :

- On peut démarrer le service publication en tapant : `npm start` (et l'arrêter en tapant `ctr+c`)
- A l'aide de Postman, Créer une requête POST en utilisant l'URL **`localhost:4000/publication/create`**
- Sous « Body », choisir l'option « row » et JSON comme type de données
- Dans la zone de texte en bas, taper un objet JSON avec un titre et un contenu :
- Après l'envoi de cette requête, on peut obtenir le résultat ci-après :



2. Créer des microservices

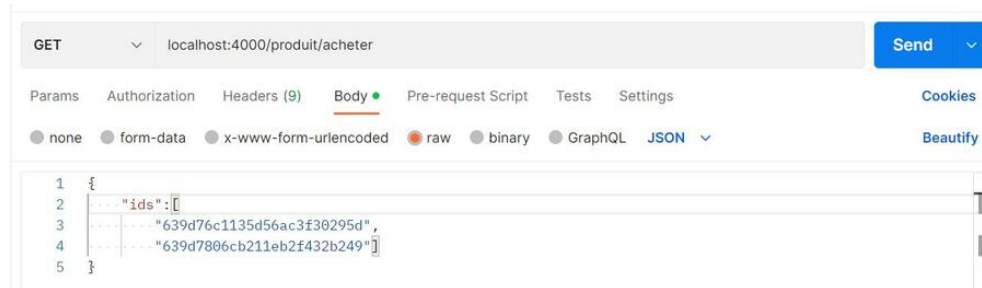
Création d'une application en microservices avec Node js : Exemple

- On suppose qu'on enregistré dans la collection Produit les deux produits suivants:

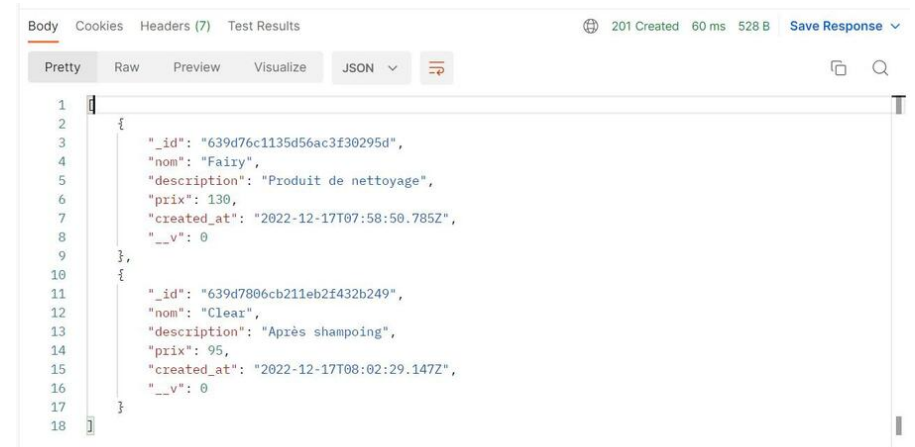
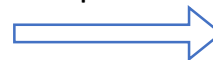
```
{
  "_id": "639d76c1135d56ac3f30295d",
  "nom": "Fairy",
  "description": "Produit de nettoyage",
  "prix": 130,
  "created_at": "2022-12-17T07:58:50.785+00:00",
  "__v": 0
}
```

```
{
  "_id": "639d7806cb211eb2f432b249",
  "nom": "Clear",
  "description": "Après shampoing",
  "prix": 95,
  "created_at": "2022-12-17T08:02:29.147+00:00",
  "__v": 0
}
```

- Si on possède leur ids et on souhaite avoir plus de détails afin de les acheter, on doit appeler la méthode de l'API **localhost:4000/produit/acheter** en lui passant les deux ids comme paramètre :



Voici le résultat de la requête :



2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple



4. commande-service :

- Sous le dossier « commande-service », refaire les mêmes étapes a, b et c de la création du service produit
- Ajouter le fichier Commande.js, à la racine de ce dossier, permettant de créer le schéma de la collection « commande » :

```
const mongoose = require("mongoose");

const CommandeSchema = mongoose.Schema({
  produits: {
    type: [String]
  },
  email_utilisateur: String,
  prix_total: Number,
  created_at: {
    type: Date,
    default: Date.now(),
  },
});

module.exports = Commande = mongoose.model("commande", CommandeSchema);
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple



4. commande-service :

c. Pour ajouter une commande, il faut fournir les identifiants des produits à commander et de récupérer le prix de chacun;

Pour ce, le service commande enverra une requête http, au service produit, pour avoir plus de détails de chaque produit en se basant sur son id;

Axios est un client HTTP basé sur des promesses pour le navigateur et Node.js. **Axios** facilite l'envoi de requêtes HTTP asynchrones aux points de terminaison REST et l'exécution d'opérations CRUD.

Pour utiliser Axios, il faut tout d'abord l'installer via la commande : **npm i axios**

Ajouter, à la racine, un fichier index.js avec le contenu ci-après :

```
const express = require("express");
const app = express();
const PORT = process.env.PORT_ONE ||
4001;
const mongoose = require("mongoose");
const Commande = require("./Commande");
const axios = require('axios');

//Connexion à la base de données
mongoose.set('strictQuery', true);
mongoose.connect(
  "mongodb://localhost/commande-
```

```
service",
  {
    useNewUrlParser: true,
    useUnifiedTopology: true,
  },
  () => {
    console.log(`Commande-Service DB
Connected`);
  }
);
app.use(express.json());
//...
```

2. Créer des microservices

Création d'une application en microservices avec Node.js



... Suite de index.js

```
//Calcul du prix total d'une commande en passant en paramètre un
//tableau des produits
function prixTotal(produits) {
  let total = 0;
  for (let t = 0; t < produits.length; ++t) {
    total += produits[t].prix;
  }
  console.log("prix total :" + total);
  return total;
}

//Cette fonction envoie une requête http au service produit pour
//récupérer le tableau des produits qu'on désire commander (en se
//basant sur leurs ids)
async function httpRequest(ids) {
  try {
    const URL = "http://localhost:4000/produit/acheter"
    const response = await axios.post(URL, { ids: ids }, {
      headers: {
        'Content-Type': 'application/json'
      }
    });
    //appel de la fonction prixTotal pour calculer le prix
    //total de la commande en se basant sur le résultat de la requête
    //http
    return prixTotal(response.data);
  } catch (error) {
    console.error(error);
  }
}

app.post("/commande/ajouter", async (req, res, next) => {
  // Création d'une nouvelle commande dans la collection
  // commande
  const { ids, email_utilisateur } = req.body;
  httpRequest(req.body.ids).then(total => {
    const newCommande = new Commande({
      ids,
      email_utilisateur: email_utilisateur,
      prix_total: total,
    });
    newCommande.save()
      .then(commande => res.status(201).json(commande))
      .catch(error => res.status(400).json({ error }));
  });
});

app.listen(PORT, () => {
  console.log(`Commande-Service at ${PORT}`);
});
```


2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple

Afin de tester l'appel de la méthode d'API **commande/ajouter**, il faut démarrer les deux services :

```
asmae@DESKTOP-PGQ50JJ MINGW64
C:\expMicroservice\produit
t-service
$ npm start

> produit-service@1.0.0 start
> nodemon index.js

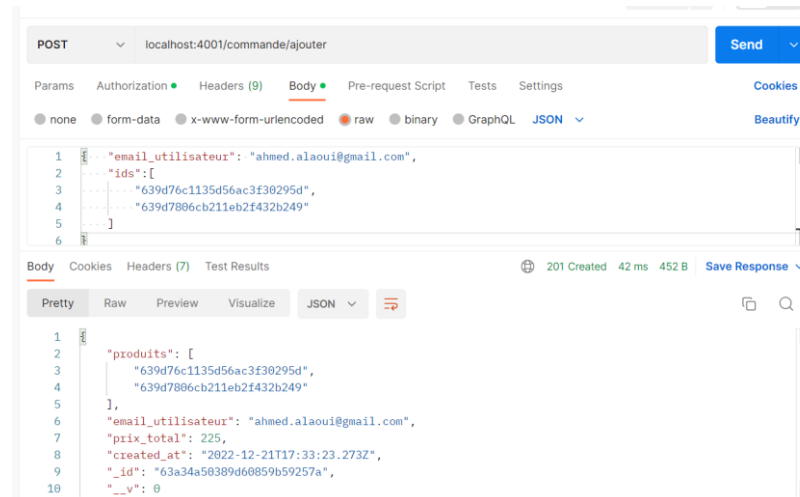
[nodemon] 2.0.20
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Product-Service at 4000
Produit-Service DB Connected

asmae@DESKTOP-PGQ50JJ MINGW64
C:\expMicroservice\commande
de-service
$ npm start

> commande-service@1.0.0 start
> nodemon index.js

[nodemon] 2.0.20
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Commande-Service at 4001
Commande-Service DB Connected
```

Sous Postman, lancer et exécuter la requête POST en lui passant deux paramètres: un tableau des ids, et un email_utilisateur



2. Créer des microservices

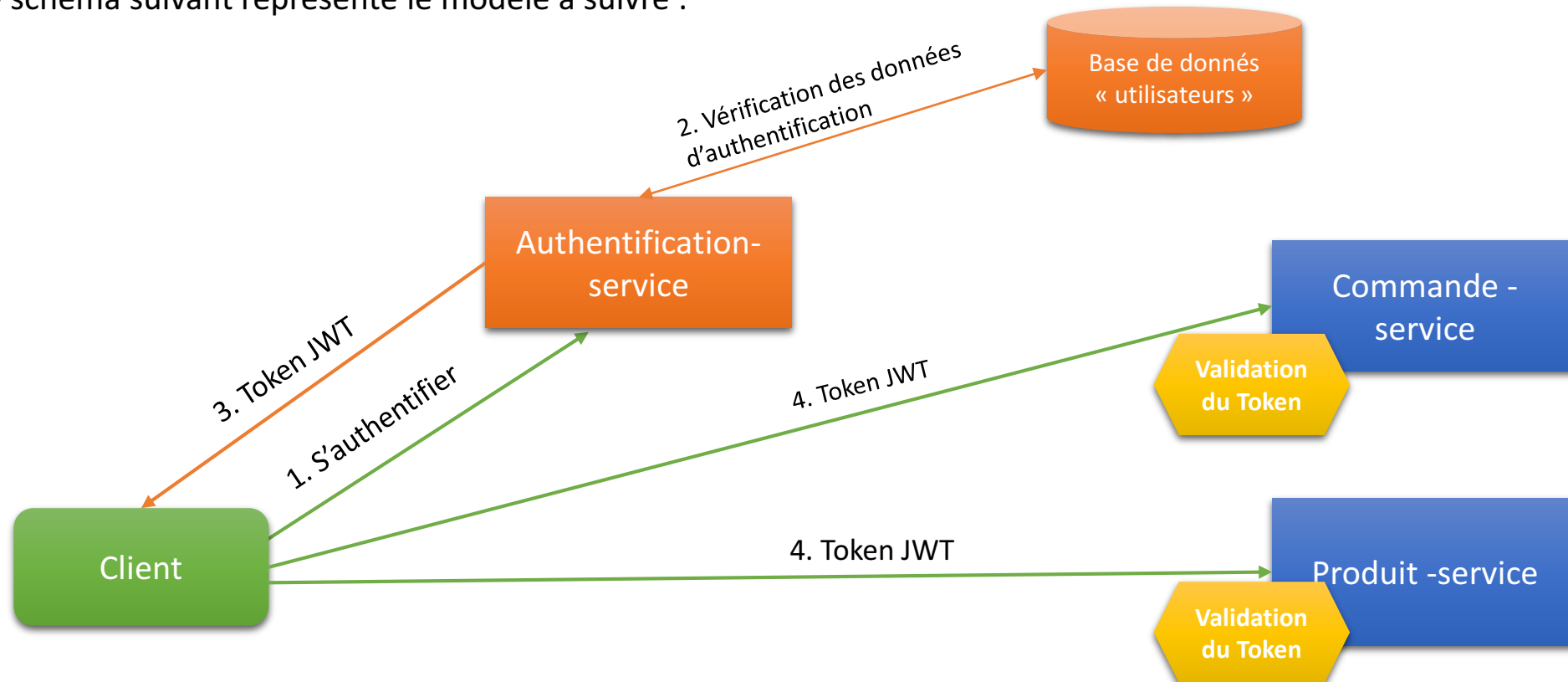
Création d'une application en microservices avec Node js : Exemple (Authentification)

Authentification :

Avant de pouvoir exploiter les différentes méthodes des API créées, un utilisateur doit être connecté.

Ainsi, on doit ajouter une étape intermédiaire d'authentification avant de consulter n'importe quel service ;

Le schéma suivant représente le modèle à suivre :



2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple (Authentification)



Authentification :

1. Le client envoie premièrement une requête au service d'authentification; En cas de réussite, celui-là crée un token JWT et l'envoie au client;

L'utilisation de JWT dans les microservices peut être comprise comme des passeports, des clés, des signes d'identification, ... pour savoir qu'il s'agit d'une demande avec une origine valide.

2. Le client sert du token pour demander l'autorisation auprès des services à exploiter.
3. Avant de répondre aux demandes auprès des utilisateurs, les services vérifient la validité du token.

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple (Authentification)



Authentification-service :

Ajoutons un nouveau service appelé « auth-service » :

1. Sous le dossier « exMicroservice », créer un sous dossier « auth-service »;
2. Initialiser npm et exécuter sous le terminal : **npm install express nodemon mongoose jsonwebtoken bcryptjs**
3. Modifier le script « test » sous package.json par : **"start": "nodemon index.js"**
4. Ajouter le schéma utilisateur.js suivant:

```
const mongoose = require("mongoose");

const UtilisateurSchema = mongoose.Schema({
  nom: String,
  email: String,
  mot_passe: String,
  created_at: {
    type: Date,
    default: Date.now(),
  },
});

module.exports = Utilisateur = mongoose.model("utilisateur", UtilisateurSchema);
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple (Authentification)

5. Ajouter le fichier index.js:

```
const express = require("express");
const app = express();
const PORT = process.env.PORT_ONE || 4002;
const mongoose = require("mongoose");
const Utilisateur = require("../Utilisateur");
const jwt = require("jsonwebtoken");
const bcrypt = require('bcryptjs');

mongoose.set('strictQuery', true);
mongoose.connect(
  "mongodb://localhost/auth-service",
  {
    useNewUrlParser: true,
    useUnifiedTopology: true,
  },
  () => {
    console.log(`Auth-Service DB Connected`);
  }
);

app.use(express.json());
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple (Authentification)

... Suite du fichier index.js

```
// la méthode regiter permettra de créer et
d'ajouter un nouvel utilisateur à la base de
données
app.post("/auth/register", async (req, res) => {
  let { nom, email, mot_passe } = req.body;
  //On vérifie si le nouvel utilisateur est déjà
  inscrit avec la même adresse email ou pas
  const userExists = await Utilisateur.findOne({
    email });
  if (userExists) {
    return res.json({ message: "Cet utilisateur
    existe déjà" });
  } else {
    bcrypt.hash(mot_passe, 10, (err, hash) => {
      if (err) {
        return res.status(500).json({
          error: err,
        });
      } else {
        mot_passe = hash;
```

```
const newUtilisateur = new
Utilisateur({
  nom,
  email,
  mot_passe
});
newUtilisateur.save()
  .then(user =>
    res.status(201).json(user))
    .catch(error =>
      res.status(400).json({ error }));
  });
});
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple (Authentification)

... Suite du fichier index.js

```
//la méthode login permettra de retourner un token
après vérification de l'email et du mot de passe
app.post("/auth/login", async (req, res) => {
  const { email, mot_passe } = req.body;
  const utilisateur = await Utilisateur.findOne({
    email });
  if (!utilisateur) {
    return res.json({ message: "Utilisateur
introuvable" });
  } else {
    bcrypt.compare(mot_passe,
utilisateur.mot_passe).then(resultat => {
      if (!resultat) {
        return res.json({ message: "Mot de
passe incorrect" });
      }
      else {
        const payload = {
          email,
          nom: utilisateur.nom
```

```

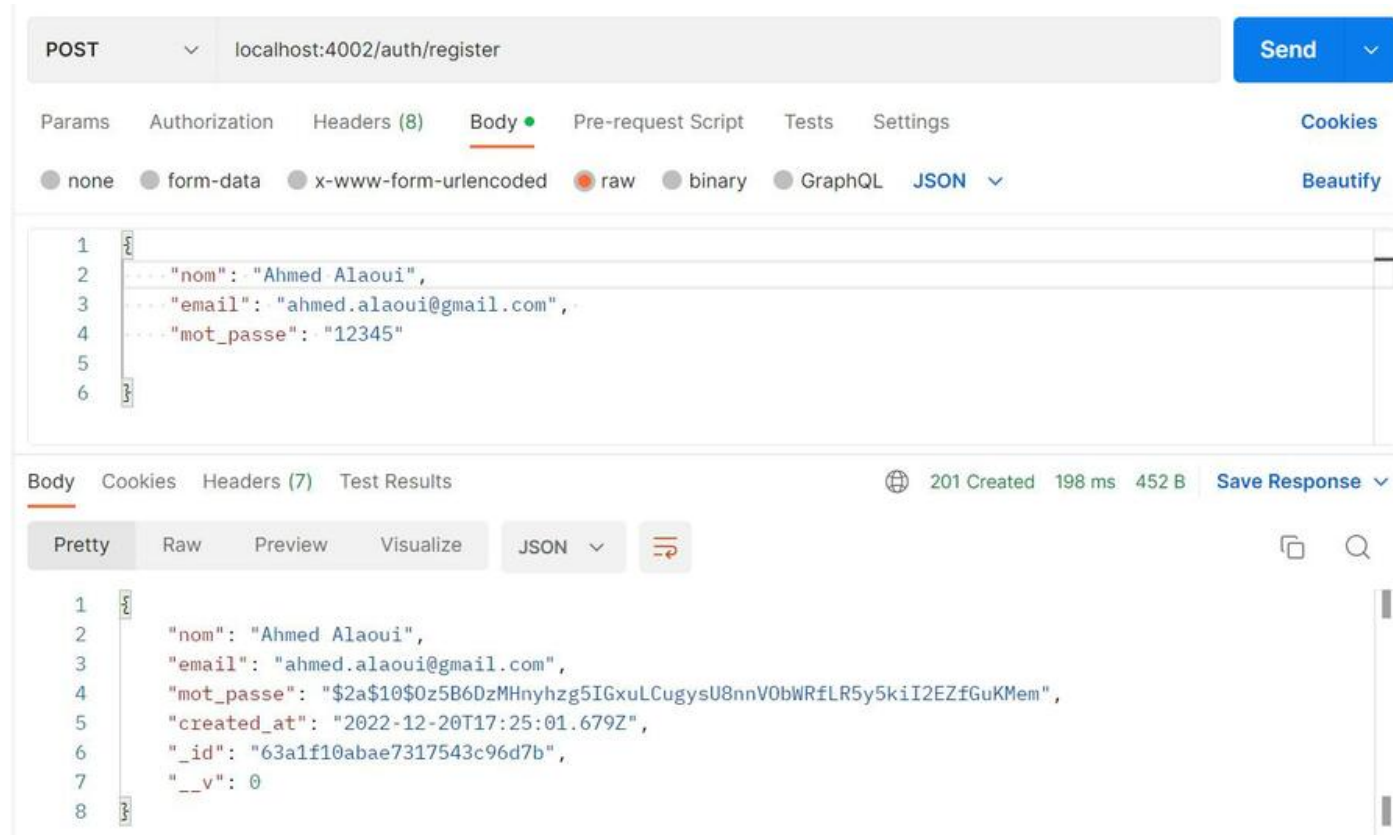
    });
    jwt.sign(payload, "secret", (err,
token) => {
      if (err) console.log(err);
      else return res.json({ token:
token });
    });
  });
});

app.listen(PORT, () => {
  console.log(`Auth-Service at ${PORT}`);
});
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple (Authentification)

Test de la méthode auth/register:



POST localhost:4002/auth/register

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "nom": "Ahmed Alaoui",
3   "email": "ahmed.alaoui@gmail.com",
4   "mot_passe": "12345"
5 }
6 }
```

Body Cookies Headers (7) Test Results 201 Created 198 ms 452 B Save Response

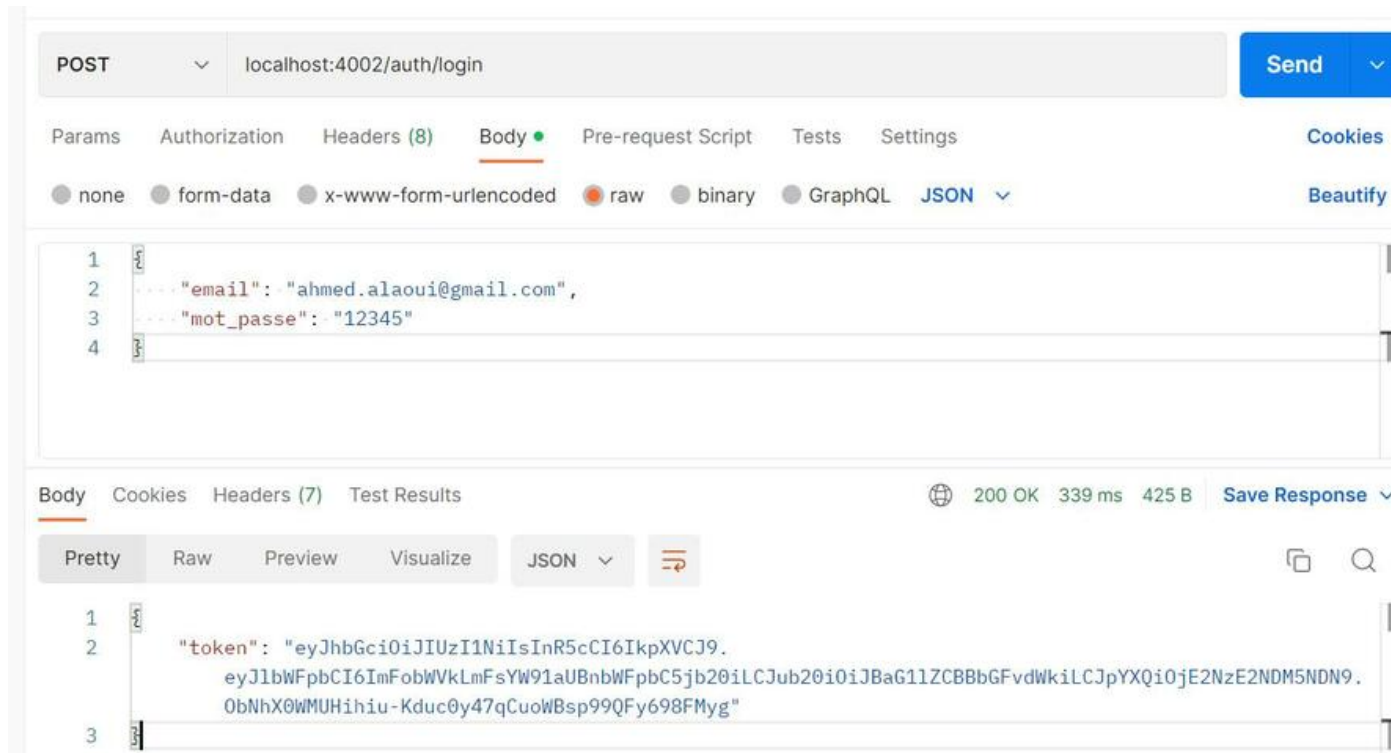
Pretty Raw Preview Visualize JSON

```
1 {
2   "nom": "Ahmed Alaoui",
3   "email": "ahmed.alaoui@gmail.com",
4   "mot_passe": "$2a$10$0z5B6DzMHnyhgz5IGxuLCugysU8nnV0bWRfLR5y5kiI2EZfGuKMem",
5   "created_at": "2022-12-20T17:25:01.679Z",
6   "_id": "63a1f10abae7317543c96d7b",
7   "__v": 0
8 }
```


Création d'une application en microservices avec Node js : Exemple (Authentification)



Test de la méthode auth/login:

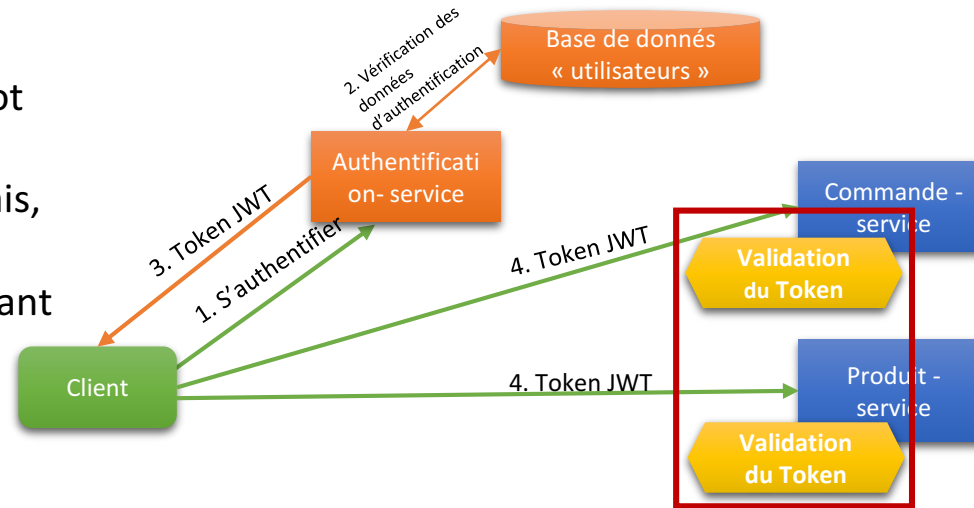


2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple (Authentification)

- Après avoir développé le service d'authentification et avoir généré le token suite à la connexion (email et mot de passe);
- Les services Produit et commande recevront, désormais, des tokens lors des appels de leurs APIs exposées.
- Ainsi, les dits services doivent valider le token reçu avant de répondre à n'importe quelle requête.

→ Ajoutons à chacun des services : Produit-service et Commande-service un fichier appelé : **isAuthenticated.js** dont le contenu est le suivant:



```
const jwt = require('jsonwebtoken');  
  
module.exports = async function  
isAuthenticated(req, res, next) {  
  const token =  
  req.headers['authorization']?.split(' ')[1];  
  
  jwt.verify(token, process.env.JWT_SECRET,  
(err, user) => {
```

```
    if (err) {  
      return res.status(401).json({ message:  
err });  
    } else {  
      req.user = user;  
      next();  
    }  
  });  
};
```

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple



→ Modifions, par la suite, nos méthodes API, pour qu'elles prennent en considération l'intermédiaire (le middleware) **isAuthenticated**; Celui là vérifiera le token avant de répondre à la requête entrante, Prenons, par exemple, la méthode **commande/ajouter** du service **commande-service**, son nouveau code se présentera ainsi :

```
app.post("/commande/ajouter", isAuthenticated, async (req, res, next) => {  
  // Création d'une nouvelle commande dans la collection commande  
  const { ids } = req.body;  
  httpRequest(ids).then(total => {  
  
    const newCommande = new Commande({  
      produits: ids,  
      email_utilisateur: req.user.email,  
      prix_total: total,  
    });  
    newCommande.save()  
      .then(commande => res.status(201).json(commande))  
      .catch(error => res.status(400).json({ error }));  
  });  
});
```

→ L'email de l'utilisateur sera récupéré par le biais du middleware **isAuthenticated** au lieu de le passer comme paramètre de la requête POST.

2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple

Pour tester cette méthode, il faut, premièrement, démarrer les trois services, de s'authentifier et de récupérer le token et deuxièmement de d'appeler la méthode commander en lui passant ce token.

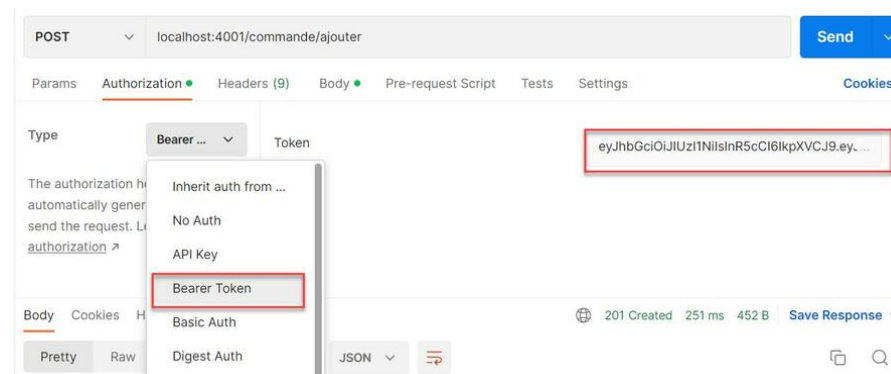
```
> produit-service@1.0.0 star
> nodemon index.js
xtensions: js,mjs,json
[nodemon] starting `node index.js`
Product-Service at 4000
Produit-Service DB Connected

> commande-service@1.0.0 sta
> nodemon index.js
xtensions: js,mjs,json
[nodemon] starting `node index.js`
Commande-Service at 4001
Commande-Service DB Connected

[nodemon] watching extension
s: js,mjs,json
[nodemon] starting `node index.js`
Auth-Service at 4002
Auth-Service DB Connected
```

Le test sous Postman nécessitera deux configurations:

1. Sous « Authorization », choisir le type « Bear » et Ajouter le token récupéré après l'exécution de la méthode auth/login du service auth-service

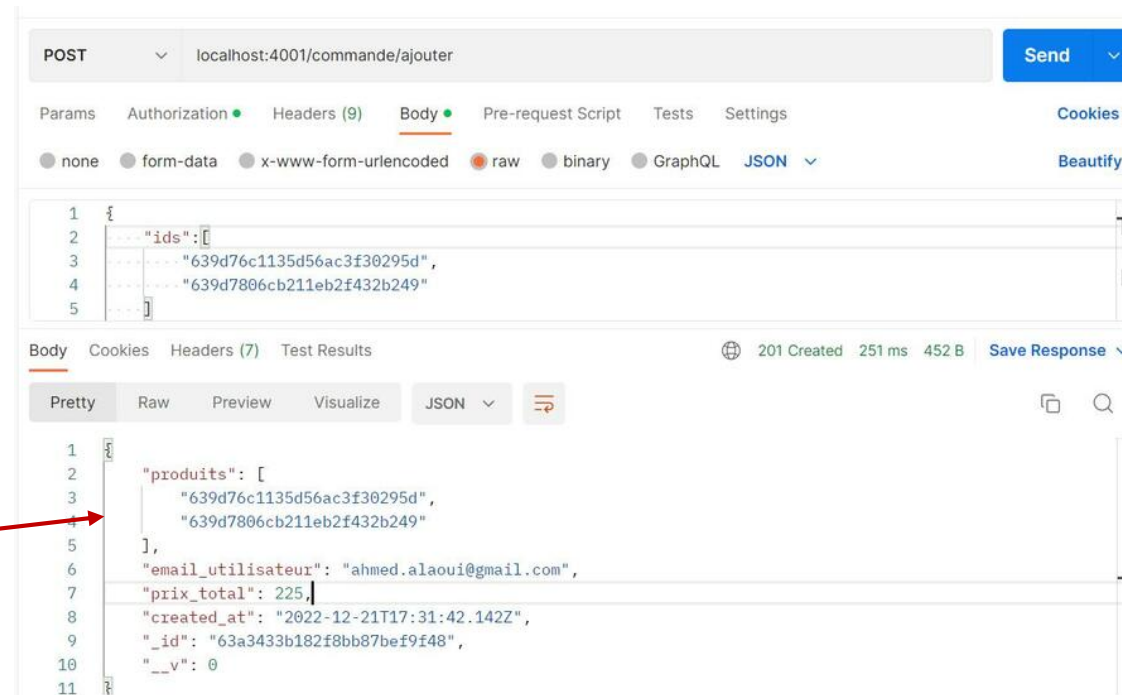


2. Créer des microservices

Création d'une application en microservices avec Node js : Exemple

2. Sous Body -> row, ajouter un tableau des ids des produits à commander

Commande
créée



L'exécution de la requête a permis de créer la commande pour l'utilisateur authentifié.

Le code de cet exemple est disponible sous le répertoire: <https://gitlab.com/asmae.youala/exp-microservice>